

Cryptography

4 – Public-key encryption: RSA

Gabriel Chênevert

September 30, 2025

JUNIA ISEN

Today

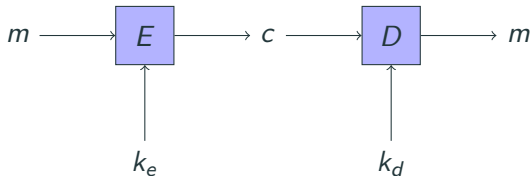
Principle

Cryptanalysis

Key generation

Asymmetric encryption

A general cipher might use *a priori* different keys for encryption and decryption:



(includes symmetric ciphers as the special case $k_e = k_d$)

if knowledge about one gives no useful information about the other

then one of them can be made public

Public-key encryption

The encryption key k_e is made public (k_d kept private)

anyone can write to Bob, but only he can read



As implemented by e.g. PGP/GPG

Famous "asymmetric" problems

- factorization of large integers

⇒ RSA

- discrete logarithm problem (DLP)

⇒ Diffie-Hellman, ElGamal, DSA

- DLP over an elliptic curve

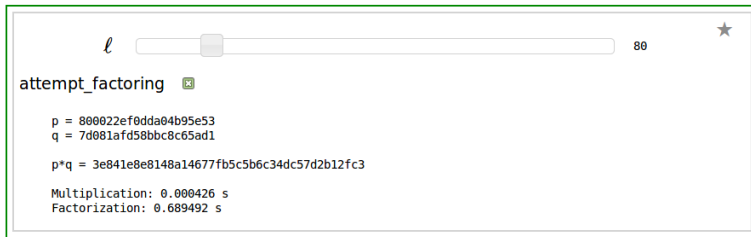
⇒ elliptic curve cryptography (ECC): ECDH, ECDSA, Ed25519, ...

- shortest vector problem / solving systems of noisy linear equations

⇒ lattice-based cryptography ...

For two ℓ -bit factors

Factorization is asymptotically *much* slower than multiplication



The screenshot shows a Jupyter Notebook interface. At the top, there is a slider control for the variable ℓ , which is set to 80. Below the slider, the text "attempt_factoring" is followed by a small icon. The output of the cell displays the prime factors p and q , their product $p*q$, and the time taken for multiplication and factorization.

```
attempt_factoring   
  
p = 800022ef0dda04b95e53  
q = 7d081afd58bbc8c65ad1  
  
p*q = 3e841e8e8148a14677fb5c5b6c34dc57d2b12fc3  
  
Multiplication: 0.000426 s  
Factorization: 0.689492 s
```

Try it for yourself

Modular arithmetic

Recall (?)

Definition

We say that $a \equiv_n b$ when n divides $b - a$, i.e. $b = a + kn$ for some integer k

i.e. a and b are equal, up to ("modulo") a multiple of n

Remarks:

- $a \equiv_n b$ if and only if $a \% n = b \% n$
- If $a \equiv_n b$ and $c \equiv_n d$, then $(a + c) \equiv_n (b + d)$ and $(ac) \equiv_n (bd)$

Rivest-Shamir-Adleman (1977)

Fix some (large) integer n .

$\mathcal{M} = \mathcal{C} = \mathbb{Z}/n\mathbb{Z}$, identified with $\llbracket 0, n\llbracket$

$$\begin{cases} E(e, m) \equiv m^e \pmod{n} \\ D(d, c) \equiv c^d \pmod{n} \end{cases}$$

based on **modular exponentiation**

Easy enough!

ℓ 20 ★

```
n = cdf45  
m = 34d78  
e = b8ee6  
m**e mod n = 16b88
```

Or is it? (try a larger ℓ)

Modular exponentiation

Naive algorithm to compute $m^e \% n$:

```
 $r = 1$   
for  $i$  in  $\llbracket 1, e \rrbracket$ :  
     $r = r * m$   
return  $r \% n$ 
```

Problems:

- intermediate result r gets LARGE
- takes e iterations

Modular exponentiation (again)

Better algorithm to compute $m^e \% n$:

```
 $r = 1$   
for  $i$  in  $\llbracket 1, e \rrbracket$ :  
     $r = (r * m) \% n$   
return  $r$ 
```

But:

- still takes e modular multiplications ...

Fast exponentiation, v.1 (R to L)

Write $e = b_\ell \cdots b_0$ in base 2, so that $m^e \equiv_n m^{b_0}(m^2)^{b_1}(m^4)^{b_2} \cdots (m^{2^\ell})^{b_\ell}$.

```
 $r = 1, q = m$   
for  $i$  in  $\llbracket 0, \ell \rrbracket$ :  
  if  $b_i = 1$ :  
     $r = (r * q) \% n$   
   $q = q^2 \% n$   
return  $r$ 
```

at most $2(\ell + 1)$ modular multiplications!

Example (v.1)

Let's compute 33^{29} modulo 227.

With $m = 33$, $n = 227$ and $e = 29 = 11101_2$:

i		0	1	2	3	4
b		1	0	1	1	1
q		33	181	73	108	87
r	1	33	33	139	30	113

so $33^{29} \equiv_{227} 113$ (indeed).

Fast exponentiation, v.2 (L to R)

Can get rid of the running variable q by writing

$$m^e \equiv_n \left(\dots \left((m^{b_\ell})^2 m^{b_{\ell-1}} \right)^2 m^{b_{\ell-2}} \dots m^{b_1} \right)^2 m^{b_0}.$$

$r = 1$

for i in $\llbracket 0, \ell \rrbracket$:

$r = r^2 \% n$

if $b_{\ell-i} = 1$:

$r = (r * m) \% n$

return r

In both cases: running time in $\mathcal{O}(\log_2 e)$

Example (v.2)

With the same values as before:

<i>i</i>		0	1	2	3	4
<i>b</i>		1	1	1	0	1
<i>r</i>	1	33	71	189	82	113

which is coherent with previous results (but uses half the memory).

Ok: that's fast

ℓ ★

```
n = cda20ec6
m = 89d3c768
e = a2f8ff3c
m**e mod n = 86df516
```

Indeed!

The RSA cipher (again)

$$\begin{cases} E(e, m) \equiv m^e \\ D(d, c) \equiv c^d \end{cases}$$

Correct decryption:

Why should there exist such exponents such that

$$m^{de} \equiv m \pmod{n} \quad \forall m \quad ??$$

Chinese Remainder Theorem

If n can be written as a product of coprime factors

$$n = n_1 \cdots n_k,$$

then there is an *isomorphism of rings*

$$\mathbb{Z}/n\mathbb{Z} \cong \mathbb{Z}/n_1\mathbb{Z} \times \cdots \times \mathbb{Z}/n_k\mathbb{Z}.$$

- $(\rightarrow)$ take remainders
- $(\leftarrow)$ use Bézout's relation

Example

$$\mathbb{Z}/12\mathbb{Z} \cong \mathbb{Z}/3\mathbb{Z} \times \mathbb{Z}/4\mathbb{Z}$$

	0	1	2	3
0	0	9	6	3
1	4	1	10	7
2	8	5	2	11

$$\text{Example: } 123^{456} \equiv_{12} ?$$

$$123^{456} \equiv_{(3,4)} \begin{bmatrix} 0^{456} \\ (-1)^{456} \end{bmatrix} \equiv_{(3,4)} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \equiv_{(3,4)} 9$$

Euler's φ function

Consider the number $\varphi(n)$ of integers in $\llbracket 0, n \rrbracket$ that are coprime with n .

Theorem (Euler)

For all x coprime with n ,

$$x^{\varphi(n)} \equiv_n 1.$$

i.e., modular exponents work modulo $\varphi(n)$: $x^a \equiv_n x^b$ when $a \equiv_{\varphi(n)} b$.

Almost there

Special case: suppose $n = p_1 \cdots p_k$ is a product of distinct prime factors, so that

$$\varphi(n) = (p_1 - 1) \cdots (p_k - 1).$$

Corollary

In this case, if $f \equiv 1 \pmod{\varphi(n)}$ then $x^f \equiv x \pmod{n} \quad \forall x$.

Hence: it is sufficient to ask that the RSA exponents satisfy

$$de \equiv 1 \pmod{\varphi(n)}.$$

A small (thus very insecure) working example

```
n   = 74989
phi = 69600
e   = 52027
d   = 10963

d*e mod phi = 1

message:  60211
encryption: 13247
decryption: 60211
```

Try here

Today

Principle

Cryptanalysis

Key generation

Security of RSA

Public: $k_e = (n, e)$, c .

The attacker would like to recover m .

- Brute force on m : search for x such that

$$x^e \equiv c \pmod{n}.$$

$\implies$ Impractical if n large

- Better: try to recover the decryption exponent d , then decrypt m like Bob

$$m \equiv \sqrt[e]{c} \pmod{n} \equiv c^d \pmod{n}.$$

Trivial if $\varphi(n)$ is known

Given e and $\varphi(n)$, the extended Euclidean algorithm easily solves

$$de \equiv 1 \pmod{\varphi(n)}.$$

But: computing $\varphi(n)$ from n is (assumed to be) hard.

Best known algorithm: **factor** $n = p_1 \cdots p_k$ and use

$$\varphi(n) = (p_1 - 1) \cdots (p_k - 1).$$

Factoring vs. splitting

Factoring n : finding the complete list of prime factors $(p_1, \dots, p_k)$ for which

$$n = p_1 \cdots p_k.$$

Splitting n : finding *one* prime factor p of n .

Essentially all known factorization algorithms are of the form

```
factors = [ ]  
while  $n > 1$ :  
     $p = \text{split}(n)$   
    factors += [  $p$  ]  
     $n = n // p$ 
```

Trial division

The simplest splitting algorithm:

```
 $p = 2$   
while  $p \leq \sqrt{n}$ :  
    if  $n \% p = 0$  return  $p$   
     $p += 1$   
return  $n$ 
```

Quickly finds small ($\leq 2^{64}$) prime factors

$\implies$ smallest prime factor should be as large as possible

$p_i \approx \sqrt[k]{n} \implies$ take $k = 2!$ (why not $k = 1?$)

$$n = p \cdot q$$

Other factorization algorithms

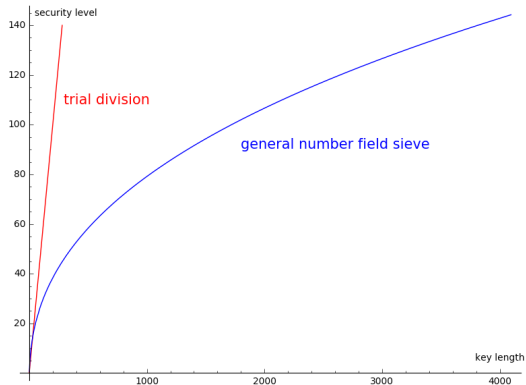
There is a **very large literature** devoted to the subject of integer factorization.

As of 2024 (and since 1995), the best general purpose algorithm is the *General Number Field Sieve* (GNFS) that factors an ℓ -bit integer in

$$\approx 5.5 \ell^{1/3} (\ln \ell)^{2/3} \text{ time.}$$

Public factorization record: **RSA-250** (2020)

Consequence on key length



According to RSA Security, Inc.

Symmetric key size	Equivalent RSA key size
80	1024
112	2048
128	3072
256	15360

Parameter reuse

Warning: Knowing (d, e) for given n is equivalent to knowing the factors of n

We know $c := de - 1$ is a multiple of $\varphi = (p - 1)(q - 1)$.

So: for every integer a we have $a^c \equiv 1 \pmod{n}$ (Fermat's theorem). Write $c = 2^t r$ where r is odd (take out powers of 2) and try different values of a until one is found for which

$$a^r \not\equiv \pm 1 \pmod{n}, \quad a^{2^t r} \equiv 1 \pmod{n}.$$

We can then factor the congruence as

$$(a^r - 1)(a^r + 1) \equiv 0 \pmod{p \cdot q}$$

and find

$$p = \text{GCD}(a^r - 1, n), \quad q = \text{GCD}(a^r + 1, n).$$

Fermat attack

Example

Suppose we have

$$n = 0x1bb85a72024a0365b78cfd0bdef8a63f943fb291899a91a713$$

$$e = 0x45d9a596bdbbc35c3c24dbacf68a588bde5cb8538a120ab07f$$

$$d = 0x94064f06cbaabbd280ec708999254b23d9a5022a0f32526f.$$

What are the factors of n ?

Solution

$\implies$ never reuse a RSA modulus

Today

Principle

Cryptanalysis

Key generation

Key generation

Recovering the decryption key should be hard for the attacker. . .

. . . but easy for Bob!

Ok since he is free to choose the prime factors of n .

Key generation: produces a RSA triple (n, d, e)

A crucial part of any asymmetric cipher in addition to the

encryption and **decryption**

algorithms.

Prime factors

To generate an ℓ -bit RSA modulus n :

- generate two random $\ell/2$ -bit prime numbers p and q
- set $n := p \cdot q$

To generate a random prime number:

- generate random integers until you get a prime!

(there are some **very fast primality tests**)

Note: density of prime numbers around x is $\approx \frac{1}{\ln x}$

RSA exponents

- Knowing p and q , compute $\varphi(n) = (p - 1)(q - 1)$
- Pick e coprime with $\varphi(n)$ (doesn't even need to be chosen randomly)
- Compute d such that

$$de \equiv 1 \pmod{\varphi(n)}$$

using the extended Euclidean algorithm (XGCD)

Real-world RSA

The *plain RSA* described above has all sorts of problems:

- malleability: $E(e, m_1) \cdot E(e, m_2) = E(e, m_1 \cdot m_2)$
- lack of randomness
- fixed size of plaintext
- ...

In practice, a suitable **padding scheme** needs to be used.

$\implies$ use a library! Or better: use a more efficient asymmetric primitive...